

Dynamic Urban Traffic Prediction

End-to-End Project Roadmap — from your current progress to a deployed system

This roadmap picks up exactly where your GFG_project repository currently stands and lays out every remaining step to a working, presentable, deployed application — a single-intersection traffic-volume predictor (XGBoost) that classifies congestion as Low / Moderate / High, served through a FastAPI backend and a React + Mapbox dashboard.

What was found in your uploaded project

- **Data:** `data/traffic_dataset.csv` and a cleaned version `traffic_dataset_clean.csv` — 4,354 rows of minute-level traffic readings (Vehicle Count, Avg Speed, Vehicle Density, Saturation Flow Rate, TSR, VLSR, Speed Factor, CI, Congestion Level, etc.) for what looks like a single intersection/camera feed.
- **Data cleaning notebook** (`src/Data_cleaning/EDA.ipynb`): loads the raw CSV, drops duplicates, fixes column names, extracts Hour from the timestamp, coerces numeric columns, fills missing values with the median, engineers `Previous_Traffic` (lag feature), drops two hard-to-use columns, and saves the cleaned CSV.
- **Visualization notebook** (`src/Data_visualisation/visualization.ipynb`): distribution plots (histograms) for every numeric feature, an hourly traffic bar chart, and a congestion-level count plot.
- **Project scaffolding:** `src/logger.py` and `src/exception.py` (a reusable logging + custom-exception pattern), plus `setup.py` and `req.txt` (pandas, numpy, matplotlib, seaborn, scikit-learn, catboost, xgboost).
- **README.md** already documents a clear scope: XGBoost for traffic-volume regression, a threshold-based Low/Moderate/High classifier, FastAPI + PostgreSQL backend, and a React + Mapbox frontend — future work explicitly deferred (LSTM, GNN, real-time CCTV).

Two housekeeping fixes before you go further

- **Remove the committed `venv/` folder from the repo.** A full Python installation (thousands of files) is currently tracked in git — add `venv/` to `.gitignore` and remove it from version control (`git rm -r --cached venv`).
- **Replace hard-coded Windows paths** (e.g. `C:\Users\KIIT\GFG_project\...`) in both notebooks with relative paths (e.g. `../data/traffic_dataset.csv`) so the notebooks run on any teammate's machine or in CI.

Roadmap at a Glance

#	Phase	Status
1	Data Collection & Understanding	DONE
2	Data Cleaning & Preprocessing	DONE
3	Exploratory Data Analysis (EDA)	MOSTLY DONE
4	Feature Engineering	TO DO
5	Train/Test Split & Baseline Model	TO DO
6	XGBoost Model Training	TO DO
7	Model Evaluation & Hyperparameter Tuning	TO DO
8	Congestion Classification Logic	TO DO
9	Model Packaging & Prediction Pipeline	TO DO
10	Backend API (FastAPI)	TO DO
11	Database Integration (PostgreSQL)	TO DO
12	Frontend Dashboard (React + Mapbox)	TO DO
13	Testing, Docs & Repo Cleanup	TO DO
14	Deployment	TO DO
15	Future Enhancements (stretch goals)	OPTIONAL

PHASE 1 Data Collection & Understanding

DONE

You already have a usable dataset. Before moving on, just confirm two things with your team:

- How was this dataset generated — real sensor logs, a public dataset, or synthetically simulated? You'll need one sentence on this for your report/viva.
- Does one row = one minute, for one fixed location? Confirming this matters because it determines whether you're doing single-location time-series forecasting (your current scope) rather than network-wide prediction (a GNN problem, listed as future work).

PHASE 2 Data Cleaning & Preprocessing

DONE

Already implemented in `EDA.ipynb`:

- Duplicate removal, column-name cleanup.
- Hour extracted from Timestamp.
- Numeric coercion + median imputation for Vehicle Count, Avg Speed, Vehicle Density.
- Previous_Traffic lag feature (previous minute's vehicle count).
- Dropped IR Presence and Vehicle Types Detected (both stored as messy list/string columns).

Small improvements worth making:

- Add a Day of Week feature — your README lists it as a key input but it isn't in the notebook yet. If the dataset has no date column, you can only encode within-day patterns; note this limitation in your report.
- Instead of dropping Vehicle Types Detected entirely, consider extracting a simple derived count (e.g. number of heavy vehicles mentioned) — optional, not required for the basic version.
- Move the cleaning steps out of the notebook and into a reusable `src/components/data_ingestion.py` / `data_transformation.py` pair of scripts that use your existing `logger.py` and `exception.py` — this is the natural next use for that scaffolding, and it's what will let your FastAPI backend reuse the same cleaning logic instead of duplicating it.

PHASE 3 Exploratory Data Analysis (EDA)

MOSTLY DONE

Already implemented in `visualization.ipynb`: distribution histograms for every numeric column, an hourly traffic bar chart, and a congestion-level count plot.

A few analyses will directly strengthen your model and your report:

- **Correlation heatmap** (`sns.heatmap(df.corr())`) to see which features actually relate to Vehicle Count — this tells you what to keep for modelling.
- **Vehicle Count vs. Hour** boxplot — shows the rush-hour pattern your model needs to learn.
- **Vehicle Count vs. Congestion Level** boxplot — sanity-checks that your existing labels line up with volume, since your classification step will be threshold-based on volume.
- **Class balance check** — you already printed `value_counts()` for Congestion Level; report the exact Low/Moderate/High split (imbalance affects how you should read classification accuracy later).

PHASE 4 Feature Engineering

TO DO

Build the final feature set that XGBoost will train on:

- **Cyclical time encoding**: convert Hour into $\sin(2\pi \cdot \text{Hour}/24)$ and $\cos(2\pi \cdot \text{Hour}/24)$ so the model understands that hour 23 and hour 0 are close together.
- **More lag features**: traffic 2 and 5 minutes ago, plus a rolling mean over the last 5–10 minutes, usually add real predictive power beyond a single lag.

- **Event flag:** your README lists an optional "Event" indicator — if the dataset doesn't have one, add a placeholder column of zeros now so the pipeline and API already expect it; you can wire in real event data later without changing the interface.
- **Target definition:** confirm your regression target is `Vehicle Count` (matches the README's "predicted vehicle count" framing) — keep `Congestion Level` aside as the label you'll derive from the prediction, not as an input feature (using it as an input would leak the answer).
- Save the final feature matrix as `data/features.csv` or persist it inside a scikit-learn `ColumnTransformer` pipeline object so training and serving use identical preprocessing.

PHASE 5 Train/Test Split & Baseline Model

TO DO

- Split chronologically, not randomly — since this is time-series data, train on the earlier portion and test on the later portion (e.g. first 80% of rows vs. last 20%) so you're honestly testing forecasting, not interpolation.
- Train a simple baseline first: linear regression, or even "predict = last known value" — this gives you a number to beat and a talking point for your report ("XGBoost improved MAE by X% over the naive baseline").

PHASE 6 XGBoost Model Training

TO DO

- Train `XGBRegressor` on the engineered features to predict `Vehicle Count`.
- Start with sensible defaults (`n_estimators=300`, `max_depth=5`, `learning_rate=0.05`) before tuning.
- Also train a `CatBoost` model as a comparison point — it's already in your `req.txt`, so a one-paragraph "we compared XGBoost and CatBoost, XGBoost won on MAE" is an easy way to show rigor without much extra work.
- Plot XGBoost's built-in feature importances — a nice, low-effort chart for your report/viva that shows which inputs matter most (likely `Previous_Traffic` and `Hour`).

PHASE 7 Model Evaluation & Hyperparameter Tuning

TO DO

- Compute MAE, RMSE, and R^2 on the held-out test set (all three are already listed as your project's evaluation metrics).
- Plot predicted vs. actual `Vehicle Count` over time — a single chart that visually proves the model tracks real traffic, very effective in a demo or viva.
- Tune with `GridSearchCV` or `RandomizedSearchCV` over `max_depth`, `learning_rate`, `n_estimators`, `subsample` — keep it to a modest grid, this is a supporting step, not the centerpiece.

PHASE 8 Congestion Classification Logic

TO DO

Your README is explicit that thresholds should be data-driven, not arbitrary. A clean way to do this:

- Compute the 33rd and 66th percentile of `Vehicle Count` from the training data — values below the 33rd percentile → Low, between the two → Moderate, above the 66th → High.
- Cross-check these thresholds against the existing `Congestion Level` column: if your derived labels roughly agree with the given labels, you can cite that agreement as validation in your report.
- Store the two threshold numbers alongside the saved model (e.g. in a small JSON file) so the API can load them without retraining.

PHASE 9 Model Packaging & Prediction Pipeline

TO DO

- Save the trained model with `joblib.dump()` to `model/xgb_traffic_model.pkl` (matches the `model/` folder your README already plans for).
- Write a single `predict_pipeline.py` with one function — `predict(hour, day_of_week, previous_traffic, avg_speed, waiting_time, event) -> {predicted_count,`

congestion_level}] — that loads the model once and applies the exact same feature engineering as training. This becomes the single function your FastAPI endpoint calls.

- Wrap file/model loading in your existing CustomException and log key steps with `logger.py` — this is exactly what that scaffolding was built for.

PHASE 10 Backend API (FastAPI)

TO DO

- `POST /predict` — accepts the input features as JSON, returns predicted vehicle count + congestion level.
- `GET /history` — returns recent stored predictions/readings from the database, for the dashboard to chart.
- `GET /health` — simple liveness check, useful once you deploy.
- Validate incoming requests with a Pydantic model (matching your README's feature table) so bad input fails fast with a clear error instead of crashing the model.
- Add CORS middleware early — otherwise your React frontend won't be able to call the API from a different port/origin during development.

Minimal endpoint sketch:

```
from fastapi import FastAPI
from pydantic import BaseModel
from predict_pipeline import predict
```

```
app = FastAPI()
```

```
class TrafficInput(BaseModel):
    hour: int
    day_of_week: int
    previous_traffic: float
    avg_speed: float
    waiting_time: float
    event: int = 0
```

```
@app.post("/predict")
def get_prediction(data: TrafficInput):
    return predict(**data.dict())
```

PHASE 11 Database Integration (PostgreSQL)

TO DO

- One table, e.g. `traffic_readings(id, timestamp, hour, vehicle_count, avg_speed, predicted_count, congestion_level)` — store both the raw reading and the model's prediction for it, so you can later show "predicted vs. actual" from real logged traffic, not just the test set.
- Use SQLAlchemy for the connection/ORM layer so swapping databases later (or using SQLite for local dev) needs minimal code changes.
- Load your historical CSV into this table once, as a seed script, so the dashboard has something to show immediately.

PHASE 12 Frontend Dashboard (React + Mapbox)

TO DO

- Start with a plain form: input fields matching the API, a "Predict" button, and a result card showing predicted count + a colored Low/Moderate/High badge.
- Add a line chart (e.g. Chart.js or Recharts) of recent Vehicle Count vs. predicted count, pulled from `GET /history`.
- Mapbox is explicitly optional in your README — treat it as a stretch goal: a single marker on the map colored by current congestion level is enough to demonstrate the idea without needing a full road network.

PHASE 13 Testing, Docs & Repo Cleanup

TO DO

- Remove `venv/` from git and add it, plus `__pycache__/, *.pkl, and logs/`, to `.gitignore`.

- Fix the hard-coded Windows paths in both notebooks (use relative paths or a config constant).
- Add a couple of basic tests: one that checks `predict()` returns a sane shape/type, one that hits the FastAPI `/predict` endpoint with `TestClient` and checks a 200 response.
- Update `README.md`'s "Getting Started" section once the backend/frontend actually exist, and add a short "Results" section with your final MAE/RMSE/R² numbers and one chart.

PHASE 14 Deployment

TO DO

- Backend: containerize with a simple `Dockerfile` and deploy to a free-tier host (Render, Railway, or Fly.io all support FastAPI + Postgres easily).
- Frontend: deploy the React build to Vercel or Netlify; point it at the deployed backend's URL via an environment variable.
- Keep a local `docker-compose.yml` (FastAPI + Postgres) so any teammate can run the whole stack with one command during development.

PHASE 15 Future Enhancements (stretch goals)

OPTIONAL

Already correctly deferred in your README — don't attempt these until the basic version above is fully working end-to-end:

- Real-time CCTV-based vehicle detection (would need a computer-vision component, e.g. YOLO).
- LSTM for sequential modelling, as a second model to compare against XGBoost.
- GNN-based spatial modelling — only relevant once you have data from multiple connected intersections, not a single feed.
- Richer weather features and a Bayesian/probabilistic congestion estimate.

Suggested Timeline (for a group project)

Rough pacing assuming a small team working part-time — adjust to your actual deadline.

Week	Focus
1 (done)	Data collection, cleaning, initial EDA
2	Finish EDA (correlation, boxplots) → Feature engineering → baseline model
3	XGBoost training, evaluation, tuning → congestion threshold logic
4	Model packaging → FastAPI backend with /predict endpoint
5	PostgreSQL integration → React dashboard (form + result card)
6	Charts + optional Mapbox → testing, cleanup, README, deployment

Suggested Final Repository Structure

```
dynamic-urban-traffic-prediction/
■■■ data/           # raw + cleaned CSVs
■■■ notebooks/      # EDA.ipynb, visualization.ipynb, model_training.ipynb
■■■ src/
■   ■■■ logger.py, exception.py      # already built
■   ■■■ components/                 # data_ingestion.py, data_transformation.py, model_trainer.py
■   ■■■ pipeline/                   # predict_pipeline.py
■■■ model/           # saved xgb_traffic_model.pkl + thresholds.json
■■■ backend/         # FastAPI app (main.py)
■■■ frontend/        # React dashboard
■■■ tests/
■■■ requirements.txt
■■■ .gitignore       # venv/, __pycache__/, *.pkl, logs/
■■■ README.md
```

This roadmap builds directly on what's already in your repository and your README's scope — nothing here requires re-deciding the project's direction, just executing the remaining phases in order.